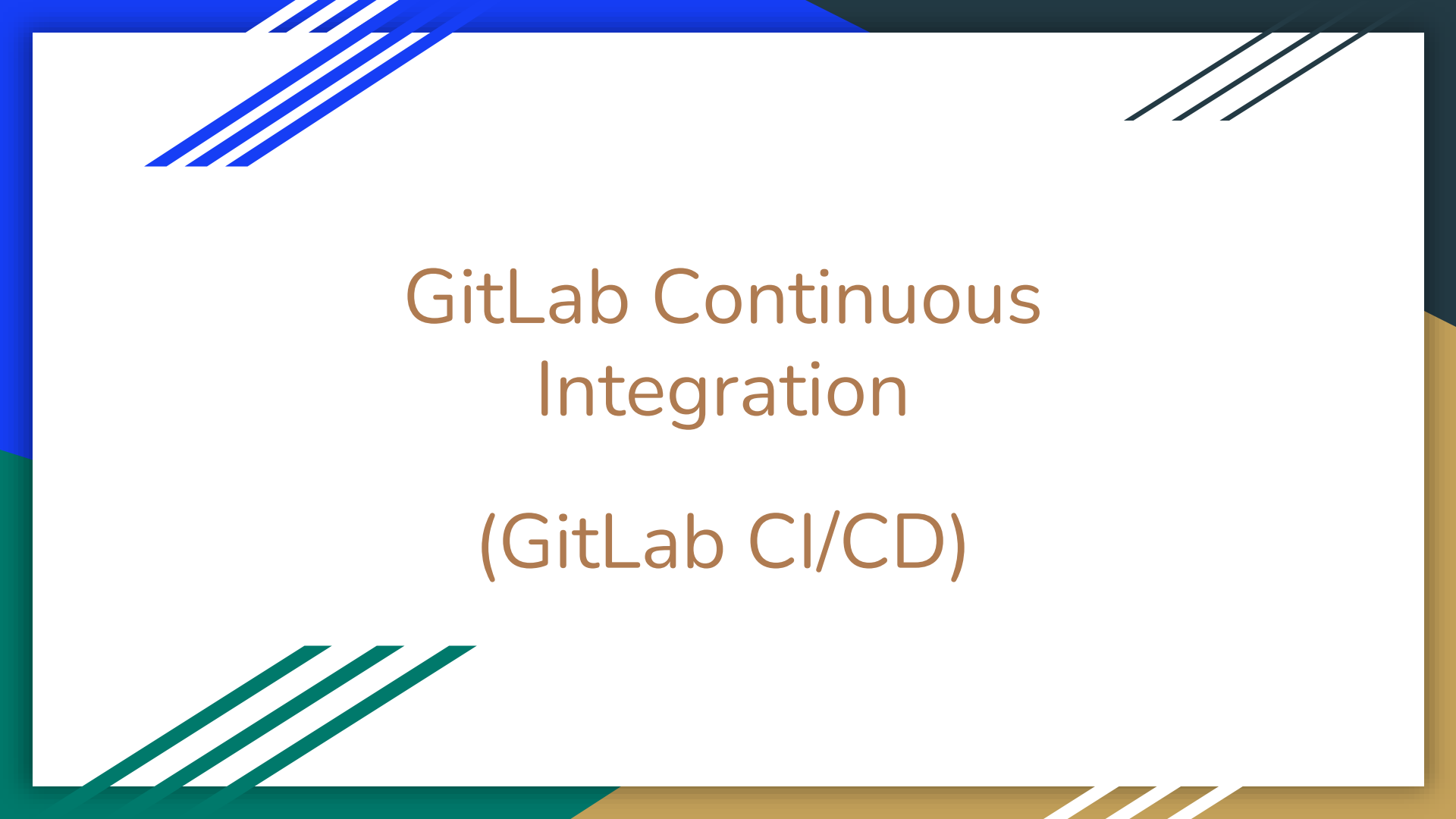




GitLab Continuous Integration (GitLab CI/CD)

CONTINUOUS INTEGRATION

- Continuous Integration is a software development practice in which you **build and test** software every time a developer pushes code to the application, and it happens several times a day.

Continuous Integration: TEST - BUILD

CONTINUOUS DELIVERY

- Continuous Delivery is a software engineering approach in which continuous integration, automated testing, and automated deployment capabilities allow software to be **developed and deployed rapidly**, reliably and repeatedly with minimal human intervention. Still, the deployment to production is defined strategically and triggered manually.

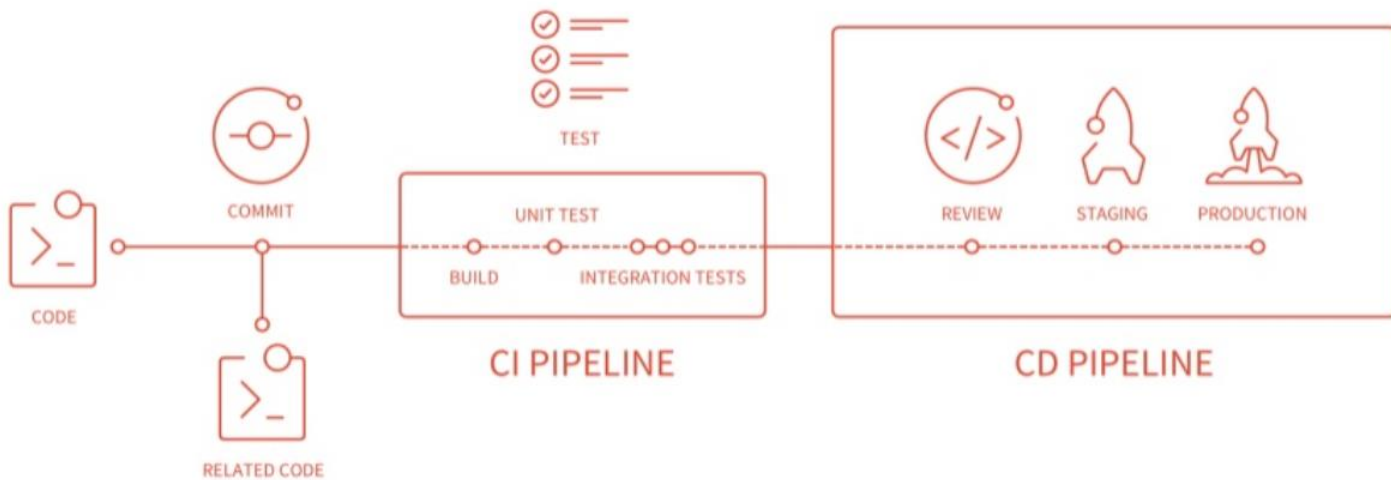
Continuous Delivery: TEST - BUILD - DEPLOY MANUALLY

CONTINUOUS DEPLOYMENT

- Continuous Deployment is a software development practice in which every code change goes through the entire pipeline and is put into production automatically, resulting in many production deployments every day. It does everything that Continuous Delivery does, but the process is fully automated, there's no human intervention at all.

Continuous Deployment: TEST - BUILD - DEPLOY AUTOMATICALLY

CI/CD PIPELINE



PIPELINE

A pipeline is a group of *jobs* that get executed in *stages*.

All of the jobs in a stage are executed in parallel (if there are enough concurrent *Runners*)

If all the jobs succeed, the pipeline moves on to the next stage. If one of the jobs fails, the next stage is not (usually) executed. You can access the pipelines page in your project's **Pipelines** tab.

STAGES

stages is used to define stages that can be used by jobs and is defined globally.

The specification of **stages** allows for having flexible multi stage pipelines. The ordering of elements in stages defines the ordering of jobs' execution:

1. Jobs of the same stage are run in parallel.
2. Jobs of the next stage are run after the jobs from the previous stage complete successfully.

JOBS

The YAML file defines a set of jobs with constraints stating when they should be run. You can specify an unlimited number of jobs which are defined as top-level elements with an arbitrary name and always have to contain at least the script clause.

Jobs are picked up by *Runners* and executed within the environment of the Runner. What is important, is that each job is run independently from each other.

Each job must have a unique name, but there are a few **reserved keywords that cannot be used as job names:**

- image, services, stages, types, before_script, after_script, variables and cache

RUNNER

GitLab Runner is the open source project that is used to run your jobs and send the results back to GitLab.

It is written in GO.

It is used in conjunction with *GitLab CI*, the open-source continuous integration service included with GitLab that coordinates the jobs.

RUNNER FEATURES

- Allows to run:
 - multiple jobs concurrently
 - use multiple tokens with multiple server (even per-project)
 - limit number of concurrent jobs per-token
- Jobs can be run:
 - locally
 - using Docker containers
 - using Docker containers and executing job over SSH
 - using Docker containers with autoscaling on different clouds and virtualization hypervisors
 - connecting to remote SSH server

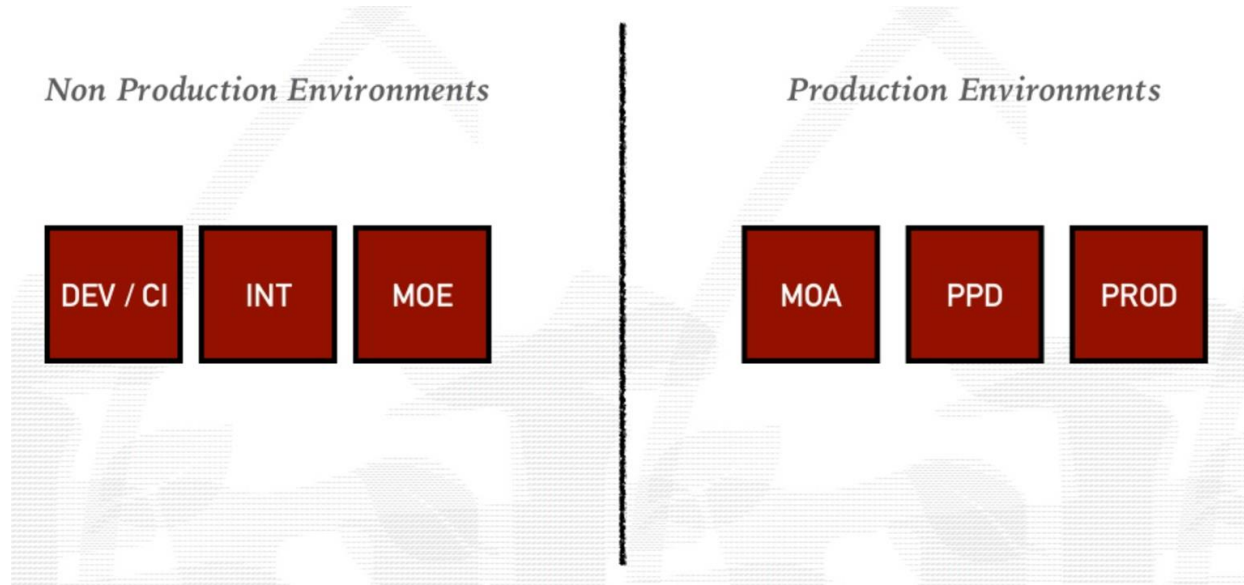
RUNNER FEATURES

- Is written in Go and distributed as single binary without any other requirements
- Supports Bash, Windows Batch and Windows PowerShell
- Works on GNU/Linux, OS X and Windows (pretty much anywhere you can run Docker)
- Allows to customize the job running environment
- Automatic configuration reload without restart
- Easy to use setup with support for Docker, Docker-SSH, Parallels or SSH running environments
- Enables caching of Docker containers
- Easy installation as a service for GNU/Linux, OSX and Windows
- Embedded Prometheus metrics HTTP server

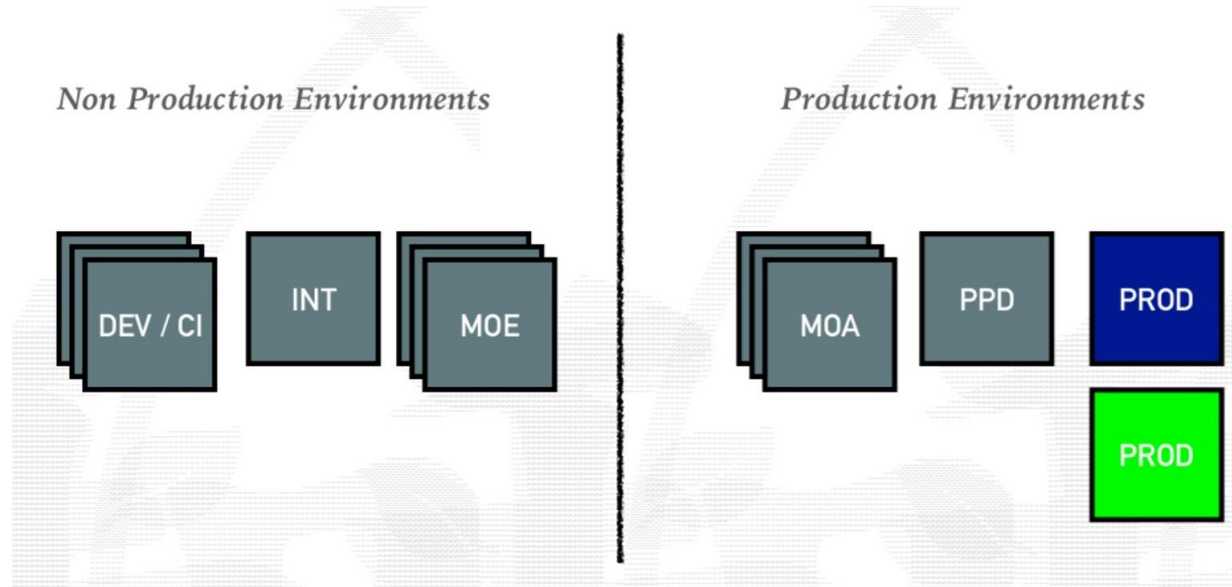


CI/CD Pipelines

CLASSICAL INFRASTRUCTURE



CLOUD INFRASTRUCTURE



Why GitLab CI?

Integration

Fully integrated with GitLab

Easy to start

A few lines in yml (YAML) inside of `.gitlab-ci.yml` and a bit clicks

Scalable

Concurrent jobs (in parallel), many runners, tagged runners

Isolated test environment

Using Docker containers

GitLab CI configuration

Is done via `.gitlab-ci.yml` file:

Example for NodeJS project:

```
nodejs_run:  
  stage: test  
  script:  
    - npm install  
    - npm test
```




Using GitLab CI

Get runner first

A simple Ubuntu Server VDS can play this role.

Provision it via script:

```
# Gitlab CI multi runner
curl -L https://packages.gitlab.com/install/repositories/runner/gitlab-ci-multi-runner/
apt-get install -y gitlab-ci-multi-runner

echo 'run "gitlab-ci-multi-runner register"'
```

Run **gitlab-ci-multi-runner** register and answer questions. You can find your unique registration token under **Settings ---> Runners** section.

Add .gitlab-ci.yml to your repo

Example for nodejs:

```
nodejs_run:  
  stage: test  
  script:  
    - npm install  
    - npm test
```



Pro tips for usage

Use cache option in your .gitlab-ci.yml to preserve dirs or files

This line will cache all git untracked files and files in node_modules dir

```
cache:  
  untracked: true  
  paths:  
    - node_modules/
```

Build skip

Build will be skipped if your commit message contains `[ci skip]`

Validate your `.gitlab-ci.yml`

Using lint: gitlab.com/ci/lint

View the status of your pipeline and jobs

When you committed your changes, a pipeline started.

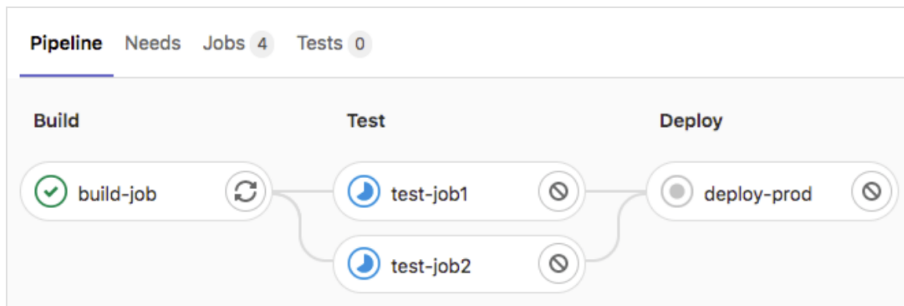
To view your pipeline:

- Go CI/CD > Pipelines.
A pipeline with three stages should be displayed:

Status	Pipeline	Triggerer	Commit	Stages
 running	#217408060 latest		 master  271ad0cd  Update .gitlab-ci.yml	  

View the status of your pipeline and jobs

- To view a visual representation of your pipeline, click the pipeline ID.

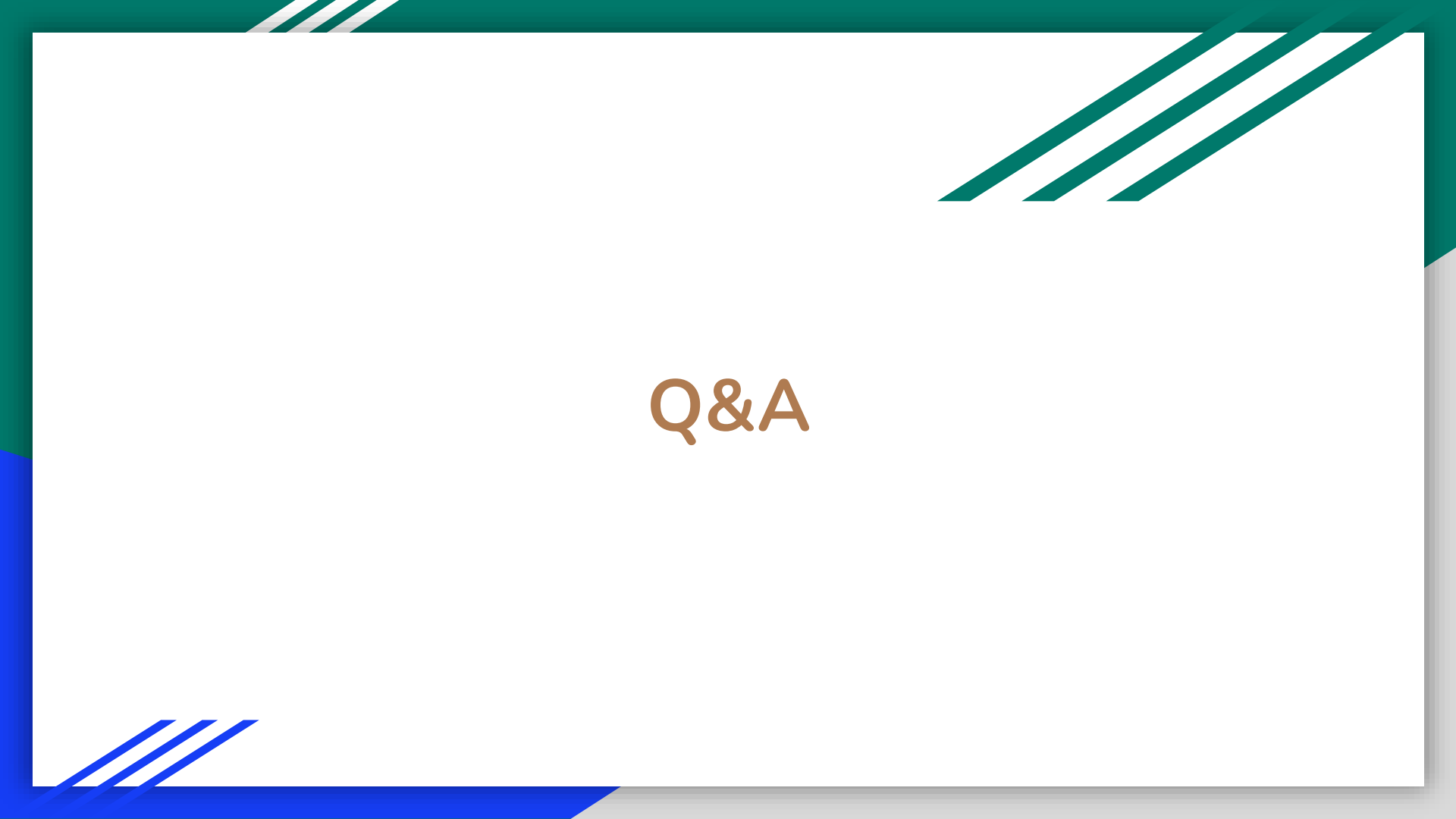


View the status of your pipeline and jobs

- To view details of a job, click the job name, for example, **deploy-prod**
- If the job status is `stuck`, check to ensure a runner is probably configured for the project.

```
✓ passed Job #855275091 triggered 23 minutes ago by 🧑 Suzanne Selhorn
```

```
1 Running with gitlab-runner 13.6.0-rc1 (d83ac56c)
2   on docker-auto-scale ed2dce3a
3   ✓ Preparing the "docker+machine" executor
4   Using Docker executor with image ruby:2.5 ...
5   Pulling docker image ruby:2.5 ...
6   Using docker image sha256:b7280b81558d31d64ac82aa66a9540e04baf9d15abb8fff
   ed62cd60e4fb5bf4132943d6fa2688 ...
7   ✓ Preparing environment
8   Running on runner-ed2dce3a-project-16381496-concurrent-0 via runner-ed2dce3a
9   ✓ Getting source from Git repository
10  $ eval "$CI_PRE_CLONE_SCRIPT"
11  Fetching changes with git depth set to 50...
12  Initialized empty Git repository in /builds/sselhorn/test-project/.git/
13  Created fresh repository.
14  Checking out 7353da73 as master...
15  Skipping Git submodules setup
16  ✓ Executing "step_script" stage of the job script
17  $ echo "This job deploys something from the $CI_COMMIT_BRANCH branch."
18  This job deploys something from the master branch.
19  ✓ Cleaning up file based variables
20  Job succeeded
```



Q&A